PROMETEO

PROGRAMACIÓN

JDBC

¿Qué es JDBC?

JDBC es una API de Java que permite la conexión y ejecución de sentencias SQL sobre bases de datos. Facilita la comunicación entre aplicaciones Java y sistemas gestores de bases de datos, permitiendo realizar operaciones como consultas, inserciones, actualizaciones y eliminaciones.

¿Porqúe es importante?

El acceso a bases de datos es fundamental en la mayoría de las aplicaciones empresariales y web. JDBC ofrece un mecanismo estandarizado y flexible para interactuar con distintos sistemas de bases de datos, favoreciendo la portabilidad y la escalabilidad de las aplicaciones.

Componentes y funcionamiento

Driver JDBC:

- Cada base de datos requiere un driver específico que implementa la API JDBC. Este driver actúa como puente entre la aplicación Java y la base de datos.

URL de Conexión:

- Especifica la dirección del servidor de la base de datos y la base de datos a la que se desea conectar. Su formato varía según el driver y el SGBD (por ejemplo, jdbc:mysql://localhost:3306/mi_base para MySQL).

Estructura

La estructura básica de una aplicación JDBC:

1. **Carga del Driver:** Se carga el driver correspondiente mediante el método Class.forName(), aunque en versiones modernas de Java, el servicio de carga automática puede encargarse de ello.
2. **Establecer la Conexión:** Se utiliza DriverManager.getConnection() pasando la URL, usuario y contraseña.
3. **Crear y Ejecutar Sentencias:** Se crea un objeto Statement o PreparedStatement para ejecutar consultas o actualizaciones.
4. **Procesar Resultados:** Con un ResultSet se recorre el resultado de una consulta y se extraen los datos.
5. **Cerrar Recursos:** Es fundamental cerrar los objetos ResultSet, Statement y Connection para liberar recursos, preferiblemente utilizando la estructura try-with-resources.

Ejemplo

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class EjemploJDBC {
    // Parámetros de conexión
    private static final String URL = "jdbc:mysql://localhost:3306/mi_base";
    private static final String USUARIO = "usuario";
    private static final String CONTRASENA = "contrasena";

    public static void main(String[] args) {
        String consulta = "SELECT id, nombre, email FROM usuarios";

        // Utilizamos try-with-resources para asegurar el cierre de recursos
        try (Connection conn = DriverManager.getConnection(URL, USUARIO, CONTRASENA);
             PreparedStatement pstmt = conn.prepareStatement(consulta);
             ResultSet rs = pstmt.executeQuery()) {

            // Procesamos los resultados
            while (rs.next()) {
                int id = rs.getInt("id");
                String nombre = rs.getString("nombre");
                String email = rs.getString("email");
                System.out.println("ID: " + id + " | Nombre: " + nombre + " | Email: " +
email);
            }
        } catch (SQLException e) {
            System.err.println("Error en la conexión o ejecución de la consulta: " + e.ge
tMessage());
        }
    }
}
```

PROMETEO

Buenas prácticas

- **Manejo de Excepciones:** Capturar y gestionar las excepciones SQLException para identificar y resolver problemas de conexión y ejecución.
- **Uso de PreparedStatement:** Para evitar inyecciones SQL y mejorar el rendimiento, es recomendable usar PreparedStatement en lugar de concatenar directamente los valores en las sentencias SQL.
- **Cierre de Recursos:** Siempre cerrar los objetos Connection, Statement y ResultSet para evitar fugas de memoria y bloqueo de recursos en la base de datos.
- **Pool de Conexiones:** En aplicaciones de gran escala, se recomienda utilizar un pool de conexiones para mejorar el rendimiento y la gestión de recursos.

JDBC en Proyectos Reales

Maven:

- Gestión de Dependencias
- Facilita añadir drivers JDBC al proyecto

Uso en patrón MVC:

- JDBC se utiliza en la capa de acceso a datos DAO

Ejemplo de flujo real:

1. Usuario interactúa con la interfaz
2. El controlador recibe la acción
3. El DAO ejecuta consultas con JDBC
4. Se obtienen datos de la base de datos
5. Se devuelven a la interfaz

Ventajas e Incovenientes

Ventajas:

- Control total
- Ligero
- Directo

Incovenientes:

- Código repetitivo
- Más complejo que ORM

Alternativas Modernas:

- Hibernate
- JPA

JDBC = base para entender ORM

Conclusión

- ★ JDBC es fundamental en Java
- ★ Permite entender cómo funciona el acceso a datos
- ★ Base para tecnologías más avanzadas (Hibernate, JPA)

PROMETEO